

AI ENGINEER · DATA · WEEK 2, DAY 3

Hybrid Retrieval, Reranking, and Filters

4 hours · instructor-led · hands-on lab

Where we are in Priya's story

- The customer asked why a spilled-biryani refund was refused
- Last week you built a corpus that can be searched at all
- Today it returns documents. Just not reliably the right ones, and never with a source Priya can quote
- By Friday, it answers with citations she can paste into the chat window

What broke last week

Acronyms retrieved nothing. Invoice numbers retrieved noise. Last year's policy outranked this year's.

Today you measure how much each new stage helps, question by question.

Keep last week's failed-query list open. It is the test set we measure against all day.

Six questions before we start

DAY 1: EMBEDDINGS, CHUNKING, INGESTION

1

What is an embedding?

2

Vector search finds what is similar. Is similar the same as relevant, or correct?

3

A scanned PDF goes into your index. What happens?

DAY 2: VECTOR STORES, RECALL, LATENCY

4

What does a vector database do?

5

What does recall@10 measure?

6

You raise efSearch from 40 to 200. What happens?

IN CHAT Pick one number and answer it in a sentence. Two minutes, then we compare.

Six questions before we start: the answers

DAY 1: EMBEDDINGS, CHUNKING, INGESTION

1

What is an embedding?

Text turned into a list of numbers, so that texts with similar meaning sit close together. BIRYANI lands near RICE, far from INVOICE.

2

Vector search finds what is similar. Is similar the same as relevant, or correct?

No. Similar is all it gives you. Relevant needs a reranker; correct needs metadata such as the withdrawn label. Both are today.

3

A scanned PDF goes into your index. What happens?

Its text comes out empty, and nothing raises an error. It needs OCR (reading text from page images) and a check that text was produced.

DAY 2: VECTOR STORES, RECALL, LATENCY

4

What does a vector database do?

Stores those lists of numbers and answers one question fast: which are nearest to this one? Approximate search trades a little accuracy for speed.

5

What does recall@10 measure?

How many of the true top 10 your search returned, divided by 10. It shows what a fast, approximate index quietly misses.

6

You raise efSearch from 40 to 200. What happens?

Recall rises, then flattens; search gets slower. efSearch can be changed in production; efConstruction needs a rebuild.

TODAY Answers 2 and 3 are this week's story: a reranker for relevance, a filter for correctness, and the scan we could not read.

Last week's five failures are today's scoreboard

QUESTION

WHY IT FAILED IN WEEK 1

YOUR BET

R01

"how many refund claims can a customer make in thirty days"

The withdrawn 2022 policy outranked the current one

?

R02

"who pays when food is spilled in transit"

The answer sits in the partner contract, in other words: "packaging inadequate for transport of liquid contents"

?

R04

"when must an agent escalate to a supervisor" The rule lives in the scanned SOP, which extracted nothing until OCR

?

R10

"CLM-2024-0314"

A claim id. Some embedding models miss it: bge-small scores it 0.32

?

R20

"TAT"

An internal acronym that no general model knows

?

ASK THE ROOM For each row, bet on one fix: keyword search, reranking, the recency filter, or none. We check the bets in the debrief.

Where we are going today

1



Hybrid retrieval

Two search methods that fail in opposite directions, combined into one.

2



Reranking

A second pass that reads the question and the candidate together, and reorders.

3



Filtering

Deciding what is even eligible to be retrieved, before similarity is considered.

4



Measure the lift

nDCG on the 20 judged questions. No claim survives today without a number.

- By the end you will have a retrieval stack that beats last week's on your own failed queries, with the numbers to prove it



1

Hybrid retrieval

Two searches that fail in opposite directions, merged into one list



Your Week 1 system is confidently mediocre



What works

Paraphrases. "Time off" finds "annual leave".
Conceptual questions land well.



What silently fails

Exact strings, product codes, acronyms, clause
numbers, anything the model never learned.



What is invisible

You cannot tell the difference from the
outside. Both return ten confident results.

A single-method retriever does not report the queries it is bad at. It returns its ten nearest neighbours every time, whether or not any of them are relevant. Silence is not the same as success.

The core issue

An analogy

Two search parties

Someone is missing at a crowded festival. You send out two search parties.

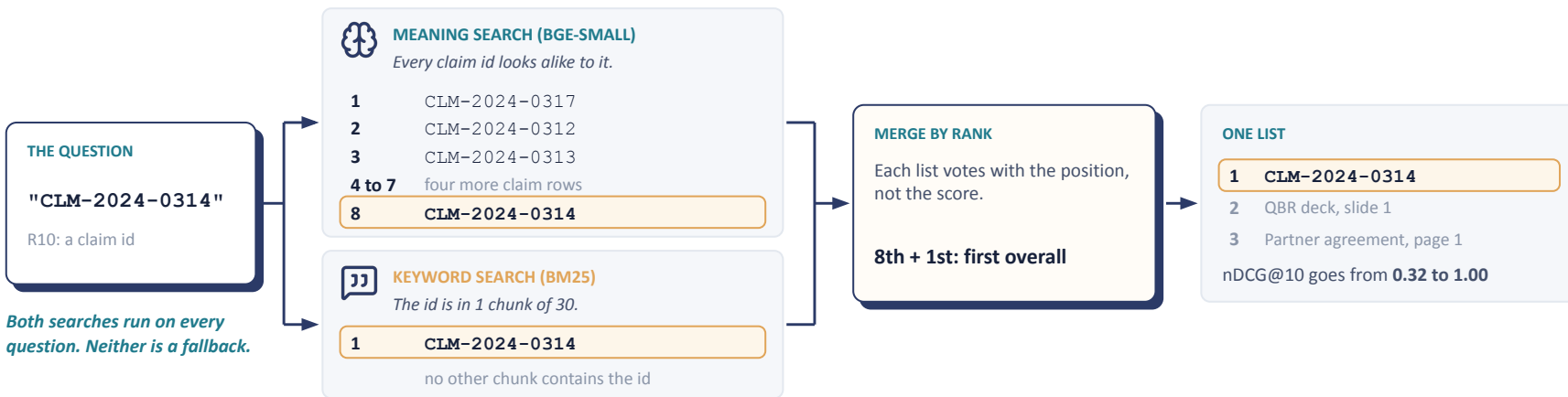
The first has a description: mid-thirties, red jacket, carrying a camera. They are excellent at finding people who look right, and they will happily bring back three strangers in red jackets.

The second has only a name badge number. They find nothing at all until they find exactly the right person.

Send one party and you get either near-misses or nothing. Send both and compare notes, and you find the person. That is hybrid retrieval.

One question, two searches, one merged list

IN ONE SENTENCE Run a meaning search and a keyword search on every question, then merge the two ranked lists into one.



MEASURED ON THE 30 LAB CHUNKS With bge-small, the open model from Lab 1, meaning search alone scores this question 0.32; merged with keyword search, 1.00.

ASK THE ROOM Which search would find "TAT"? Which one finds a question worded nothing like the document?

Keyword search: rare words count most

WHAT EACH WORD IS WORTH

"refund status for CLM-2024-0314"

Weight: how rare the word is across the 30 lab chunks.

refund	in 17 of 30 chunks	0.61
status	in 8 of 30 chunks	0.97
for	in 9 of 30 chunks	0.82
CLM-2024-0314	in 1 of 30 chunks	2.98

Best match: the CLM-2024-0314 row, score **5.16**. The next chunk scores **1.76**.

Repeats help, less each time



used once	1.00
used twice	1.43
used 10 times	2.17, not 10

Long chunks get no bonus

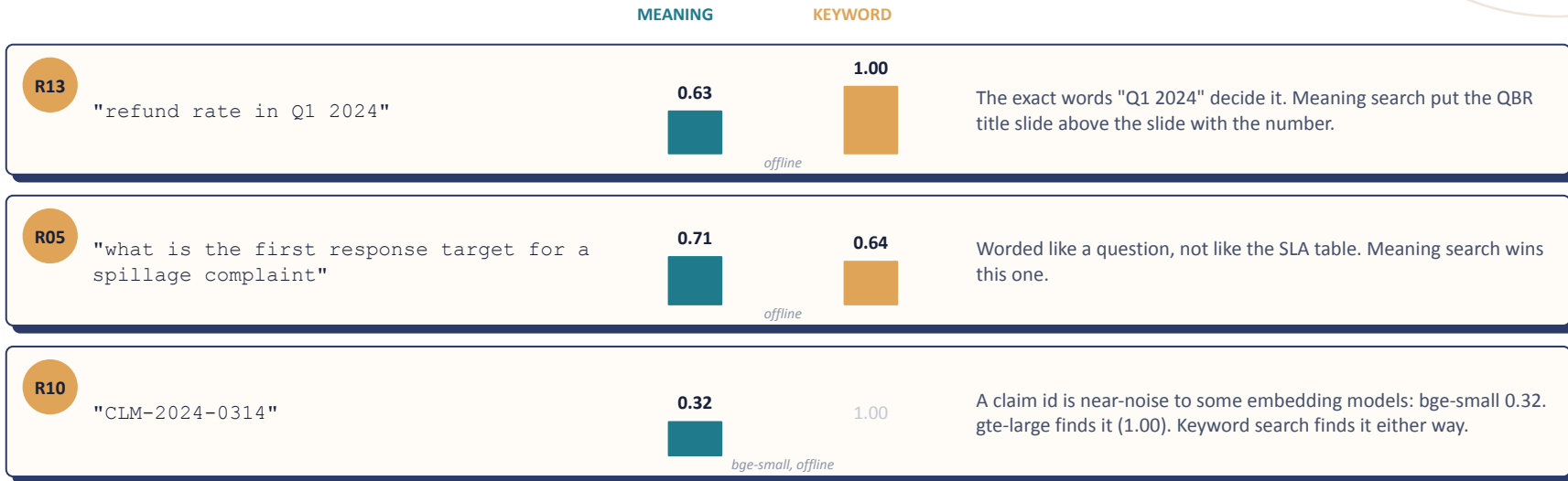


Scores are adjusted for length, so a chunk does not win just by holding more words.

WHAT IT CANNOT DO It matches words, not meaning. "spilled" is in 2 of our chunks and "spillage" in 9: to keyword search they are two different words. That gap is what meaning search fills.

ASK THE ROOM Your tickets are full of order ids like ORD-8845914. Good news or bad news for keyword search?

Meaning and keyword search miss different questions



AVERAGE OVER ALL 20 QUESTIONS, OFFLINE meaning 0.803, keyword 0.827, both fused 0.806. With real models on Databricks, meaning 0.862 and fused 0.908. Fusing is not automatically better: measure it.

ASK THE ROOM Which of your own queries would only keyword search find? Name one.

Two searches, two scales: the scores cannot be added

"refund rate in Q1 2024" (R13)

MEANING SEARCH

a similarity: always between 0 and 1

1	QBR slide 1: the title slide	0.78
2	QBR slide 2: "refund rate ... 4.2%"	0.62
3	Refund policy: document header	0.18

KEYWORD SEARCH (BM25)

a word-match score: no fixed top, 7.66 on this question

1	QBR slide 2: "refund rate ... 4.2%"	7.66
2	QBR slide 1: the title slide	4.51
3	QBR slide 3: escalation performance	3.22

ADD THE RAW SCORES, AND KEYWORD SEARCH DECIDES ALONE

QBR slide 2 (the answer)	$0.62 + 7.66 = 8.28$
QBR slide 1 (the title)	$0.78 + 4.51 = 5.29$
QBR slide 3 (escalations)	$0.00 + 3.22 = 3.22$

The sum keeps the keyword order exactly.

ASK THE ROOM Why not divide the keyword scores by 10 first, then add them?

Fusion by rank, worked on one real question

"refund rate in Q1 2024" (R13)

MEANING SEARCH

1 QBR slide 1: the title slide

2 QBR slide 2: "refund rate ... 4.2 percent"

3 Refund policy: document header

KEYWORD SEARCH (BM25)

1 QBR slide 2: "refund rate ... 4.2 percent"

2 QBR slide 1: the title slide

3 QBR slide 3: escalation performance

Scores from the two searches are on different scales (0.78 against 7.66 here), so they cannot be added. Rank can. Being high in both lists wins; being in only one still counts.

SCORE = 1 / (60 + RANK)

added up across both lists

Title slide

$$1/61 + 1/62 = 0.0325$$

Answer slide

$$1/62 + 1/61 = 0.0325$$

A tie. The answer stays at number 2, so nDCG@10 is 0.631.

AFTER THE BREAK A reranker reads the question and each slide together, puts the answer first, and nDCG@10 goes from 0.631 to 1.000. Same two documents, better order.

ASK THE ROOM A chunk appears only in the keyword list, at rank 1. Does it beat one that is rank 3 in both lists?

When to lean one way or the other

Leaning: giving one list more weight when the two are merged.



Lean keyword

Corpora dense with identifiers, part numbers, clause references, error codes.

Legal, manufacturing, support ticketing.



Balanced start

Most enterprise document sets.

Start equal, measure on your own queries, adjust once. Do not tune before you can measure.



Lean semantic

Conversational corpora where users ask in their own words and documents are written in someone else's.

Policies, wikis, FAQs.

Weighting is a per-corpus decision, not a universal one. Anyone quoting a magic ratio has not seen your data.

Before the break, three questions

1

A user searches for "status of CLM-2024-0314" and gets three documents about claims, none mentioning CLM-2024-0314. Which retrieval method failed, and which one would have caught it?

2

Why can you not simply add a cosine similarity score and a BM25 score together to rank results?

3

Your team proposes running keyword search only when vector search "returns nothing". Why will that never trigger?

Take answers in chat. Do not move on until at least two people have committed to an answer.

Before the break, three questions

1

The claim id miss

Dense retrieval failed: the model never learned what CLM-2024-0314 means, so it embedded close to noise and the nearest neighbours were generic claims text. Keyword search treats it as a rare exact token and ranks it first.

2

Why scores cannot be added

They are on different, incomparable scales, a cosine similarity near 0.8 and a BM25 score of 14 measure different things. Reciprocal rank fusion sidesteps this by using position in each list rather than the raw score.

3

The fallback that never fires

Vector search always returns its k nearest neighbours, however poor they are. It never returns nothing, so the fallback condition is never met. The two methods have to run together on every query.

Break

10 minutes. After the break: reranking, what it improves and what it costs.



2

Reranking

The second opinion that reads the question properly



Search ranks what looks like the question, not what answers it

"refund rate in Q1 2024" (R13)

SEARCH PUT THIS FIRST

QBR slide 1: "Support Operations QBR, Q1 2024. Prepared for the Support Leadership Review, 12 April 2024."

Shares the words "Q1 2024". Answers nothing.

SEARCH PUT THIS SECOND

QBR slide 2: "Refund volume and rate. ... so the refund rate fell slightly from 4.4 percent to 4.2 percent."

Holds the answer: 4.2 percent.

WHY Search turned the question and every chunk into numbers separately, in advance, and only compares how alike they look. It never reads a question and a chunk together, so it cannot tell which one answers.

The right chunk was found, just in the wrong place: nDCG@10 for this question is 0.631, not 1.000.

ASK THE ROOM You can rank these two correctly at a glance. What are you doing that the search is not?

An analogy

The shortlist and the interview

Two thousand CVs arrive for one role. Nobody reads two thousand CVs.

So a keyword screen cuts it to fifty in seconds. That screen is crude and everyone knows it, but it is fast and it is roughly right.

Then you interview the fifty properly, the candidate and the job description in the room together, and you rank them on how well they actually fit.

You would never interview two thousand people. You would also never hire from the keyword screen alone. Retrieval works the same way: search wide and cheap, then judge narrow and well.

Reranker, in one sentence

A model that looks at the question and one candidate chunk together, and scores how well that chunk actually answers that question.

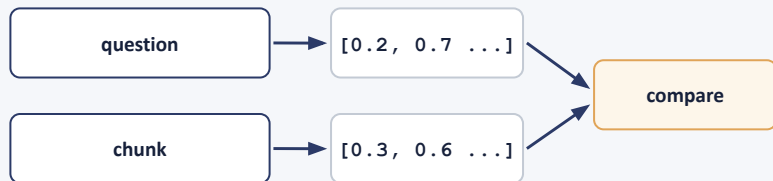
- Your embedding model never saw the question and the document at the same time, it encoded each separately, in advance
- A reranker reads both together, which is why it judges relevance far more accurately
- It is too slow to run over a whole corpus, so it only ever runs on the shortlist retrieval produced

Retrieve 50 candidates, rerank to the best 5, send those to the model.

Why a reranker judges better than the search

Bi-encoder: the search

Question and chunk are turned into numbers separately



Chunks are encoded once, in advance, so millions can be searched in milliseconds. The two texts never meet.

Cross-encoder: the reranker

Question and chunk go through the model together



Every word of the question can attend to every word of the chunk, so it judges relevance, not resemblance. One model call per candidate, at question time.

IN LAB 3 Offline, the reranker is a stand-in built from word statistics. On Databricks, Llama 3.3 70B grades each passage 0 to 3: 3.7 s a question at the median, against 0.23 s for hybrid search.

ASK THE ROOM Why can you not run the reranker over the whole corpus instead of the shortlist?

Retrieve wide and cheap, then judge narrow and well

1,000,000 chunks FreshCart at full size: 40,000 files

50 candidates Hybrid search, milliseconds. Tune it for recall: you cannot rerank what you never found.

top 5 Reranker, one call per candidate. Tune it for precision.

the prompt Only these reach the model

STAGE 1 SETS THE CEILING If the right chunk is not in the 50, no reranker can rescue it. Measure recall@50 of the search before you tune the reranker.

ASK THE ROOM Rerank 50 or 100 candidates? Say what improves and what it costs.

What reranking costs you



Latency

One model pass per candidate. Reranking 50 typically adds tens to low hundreds of milliseconds.

Batch them; do not loop one at a time.



Money

Hosted rerankers charge per document scored. 50 candidates per query at scale is a real line item.

Smaller open models run locally and are often good enough.



Complexity

Another model to version, monitor and evaluate.

It can also degrade quality if the reranker is poorly matched to your domain. Measure, do not assume.

Reranking is usually worth it. "Usually" is a claim you should verify on your own queries today, not accept from me.

Rerankers and hybrid search: the names you will meet

Rerankers

HOSTED

Cohere Rerank

Voyage rerank

Jina Reranker

OPEN, RUN IT YOURSELF

bge-reranker

ms-marco cross-encoders

BUILT INTO A SEARCH SERVICE

Azure AI Search semantic ranker

A MODEL AS THE JUDGE

gpt-4.1-mini grading 0 to 3 (our Azure track)

Hybrid search built in

MANAGED SEARCH

Azure AI Search (RRF built in)

Elasticsearch

OpenSearch

DATABASES

PostgreSQL: pgvector + full-text

Databricks AI Search

VECTOR DATABASES

Weaviate

Qdrant

Milvus

Chroma (in its cloud only)

BY HAND (OUR OFFLINE TRACK)

rank_bm25 + our own RRF

HOW TO CHOOSE Check three things on your own questions: the nDCG lift, the added time per question, and the cost per 1,000 questions. Lab 3 gives you the first two.

ASK THE ROOM Your data may not leave the network. Which of these can you still use?

Measuring whether it helped

Did the good results end up near the top, where a user or a model will actually see them?

- Recall asked: was the right chunk retrieved at all? nDCG asks: was it ranked highly?
- A result in position 1 counts far more than the same result in position 9. That discount is the whole idea
- It needs relevance judgements, someone deciding which chunks genuinely answer which query
- Those judgements are the expensive part, and they are the asset. Build them once, reuse them all programme

Reranking barely changes recall, because it reorders what retrieval already found. It changes nDCG a lot. Measure the right thing or you will conclude reranking does nothing.

The trap

A grade, a rank, and the best possible order

1 Grade: how good is it?

Set in advance by a person, in the golden file. Then turned into points.

grade 3	answers the question	7 pts
grade 2	relevant	3 pts
grade 1	related	1 pt
grade 0	anything else	0 pts

2 Rank: where did it land?

Its position in the result list, 1 at the top. Lower down keeps less of its points.

rank 1		keeps 100%
rank 2		keeps 63%
rank 3		keeps 50%
rank 4		keeps 43%
rank 6		keeps 36%
rank 10		keeps 29%

$$\text{share kept} = 1 / \log_2(\text{rank} + 1)$$

3 Score: compare with the best

Add up the top 10, then divide by what the perfect order would earn.

points earned

grade points x share kept, added up over the top 10

best possible

the same graded chunks, best first, from rank 1

nDCG@10

points earned / best possible

1.0 = best answers on top

R02 FROM LAB 3 grade 3 at rank 6: $7 \times 36\% = 2.49$; grade 1 at rank 4: $1 \times 43\% = 0.43$. Earned 2.92. Best possible: $7 \times 100\% + 3 \times 63\% + 1 \times 50\% = 9.39$. $\text{nDCG@10} = 2.92 / 9.39 = 0.311$.

ASK THE ROOM A question has one good answer, and it sits at rank 2. What is its score? What if it moves to rank 1?

nDCG on one question: answer at 2, then at 1

AFTER FUSION

1	grade 0	x 1.00
2	the answer slide, grade 3	x 0.63
3	grade 0	x 0.50
4	grade 0	x 0.43
5	grade 0	x 0.39

nDCG@10 = 0.631

AFTER RERANKING

1	the answer slide, grade 3	x 1.00
2	grade 0	x 0.63
3	grade 0	x 0.50
4	grade 0	x 0.43
5	grade 0	x 0.39

nDCG@10 = 1.000

HOW IT IS COUNTED

A grade-3 answer is worth 7 points.
Each position down divides it by a bit more.

At 2: $7 \times 0.63 = 4.42$
Best possible: 7
 $4.42 / 7 = 0.631$

THE TRAP Recall@10 is identical in both columns: the same chunks were found. Only the order changed. Measure recall and you conclude the reranker did nothing.

ASK THE ROOM Where do the grades come from, and who in your team would write them?



3

Filtering

Deciding what is even allowed to be retrieved



The withdrawn policy wins, because it says almost the same thing

"how many refund claims can a customer make in thirty days" (R01)

SEARCH RANK 1, SIMILARITY 0.94

Refund policy 2022: "A customer may make up to two refund claims in any rolling thirty day period."

Withdrawn. Says two.

SEARCH RANK 2, SIMILARITY 0.80

Refund policy 2024: "A customer may make up to three refund claims in any rolling thirty day period."

In force. Says three.

WHY Search scores how alike the words are. The two versions are almost word for word the same, and nothing in the text says which one was withdrawn. That fact lives outside the text.

An answer built on the top result tells the customer two claims. The rule in force is three.

ASK THE ROOM Would a better search model fix this? If not, what would?

The filter is a correctness control, not an optimisation



Recency

Superseded policies must not compete with current ones. This fixes last week's refund failure, where last year's policy outranked this year's.



Entitlement

Priya sees her customer. Not every customer. Enforced here, never by asking the model nicely.



Scope

Region, business unit, document type, language. Narrowing the field usually raises quality as well as safety.

If a chunk reaches the context window, treat it as disclosed. There is no reliable way to show a model a document and instruct it to keep the contents secret.

Filter before you search, or lose results without an error

Post-filter

Search everything, then drop what the user may not see.



Asked for 10, got 0 to 3, under 1 on average. 9 of the 20 questions got nothing. Reported as success.

Lab 2, step 6: only 1 vector in 10 carried the allowed label.

Pre-filter

Search only inside what the user may see.



Asked for 10, got 10, on all three databases. Costs more; some stores slow down sharply.

The same 20 questions, on Chroma, Qdrant and Milvus.

TODAY'S FILTER The recency filter drops the withdrawn 2022 policy: 7 of the 30 chunks. In Lab 3 it is the biggest single gain offline, +0.059. With real models it adds +0.034 and is the only stage that fixes R01.

ASK THE ROOM An entitlement filter hides 95 percent of the corpus from a new joiner. What does post-filtering show her?

Three questions before the lab

1

Your reranker makes no difference to recall@10 . Your manager concludes it is not working. What would you measure instead, and why?

2

You retrieve the top 10 and rerank them. A colleague suggests retrieving the top 100 instead. What improves, and what does it cost?

3

An entitlement filter removes 95 percent of the corpus for a given user. Which filtering design would you insist on, and what breaks with the other one?

Take answers in chat. Do not move on until at least two people have committed to an answer.

Three questions before the lab

1

The reranker that changed nothing

Measure nDCG, not recall. Reranking reorders what retrieval already found, so recall is largely unchanged by design. Ranking quality is the thing that moved, and it is what determines which chunks reach the model.

2

Retrieving 100 instead of 10

Recall at stage 1 improves, so the reranker has more chance of finding the right chunk, you cannot rerank what you never retrieved. The cost is latency and money, since the cross-encoder now scores 100 candidates per query. Measure where the curve flattens.

3

The 95 percent filter

Pre-filtering. With post-filtering the search returns 10, then 9 or 10 are discarded, and the user gets an almost empty answer with nothing reporting a fault. Pre-filtering restricts the candidate set first and returns the right number of results.

Day 3: before you build the stack

Can we skip keyword search if the embedding model is good?

No. Exact ids and rare strings are where embeddings are weakest, and models differ. On meaning search alone the claim id scores 0.32 with bge-small, 0.50 with text-embedding-3-small and 1.00 with gte-large.

Should keyword search be the fallback when vectors find nothing?

That fallback never fires: vector search always returns its nearest neighbours, however poor. Run both on every question.

Is reranking worth the wait?

Measure it. With real models on Databricks it lifts nDCG@10 from 0.908 to 0.953 and takes about 3.7 seconds a question. If that is too slow, rerank fewer candidates before you drop the stage.

How many chunks do we send to the model?

Retrieve 30 to 100, rerank, keep 3 to 8. Fewer, better chunks beat more, noisier ones. Lab 4 keeps 5.

Can we tell the model not to reveal restricted documents?

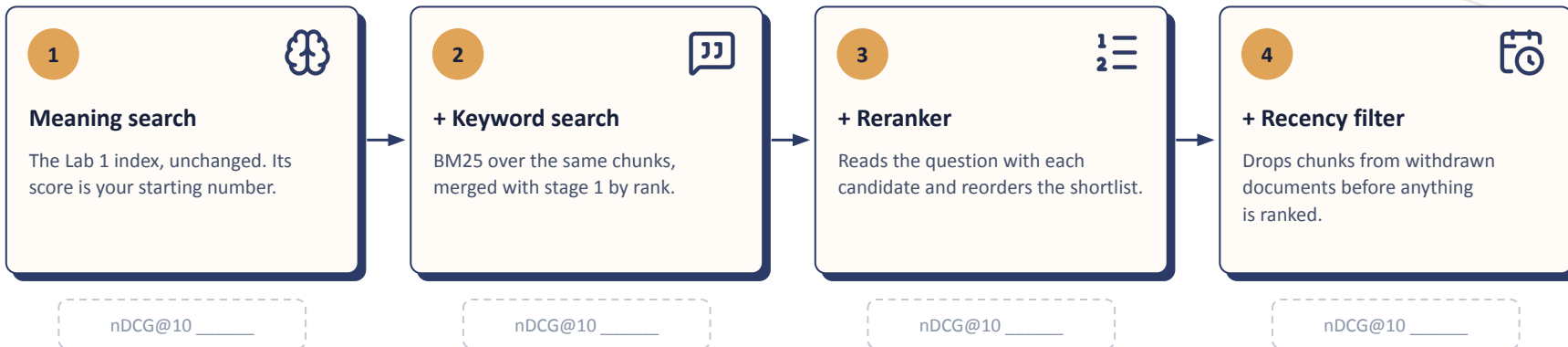
No. A chunk that reaches the prompt is disclosed. Permissions are a filter that runs before the search, never an instruction to the model.

Who writes the relevance grades?

People who know the answers, such as a support lead, not the engineer alone. A small set judged carefully beats a large one generated by a model. Lab 3 hands you 20.

One more, and it decides what you build next: which stage moved the number most? Measure one stage at a time, or you will never know.

Four stages, measured one at a time



AFTER EVERY STAGE Score the same 20 judged questions with nDCG@10, and time each one. Change one thing at a time, or a single number cannot tell you which change earned its place.

YOU HAND IN The four scores and times, one question that got worse and your theory why, and the one addition you would ship.

ASK THE ROOM Before you run it: which of the five Week 1 failures will still fail after stage 4?

Lab 3

Add a reranker and a metadata filter, and measure the lift

75 minutes · same pairs · your Week 1 corpus · lab03_hybrid notebook

Build

- Add BM25 alongside your vector search
- Fuse the two result lists with RRF
- Add a reranker (a stand-in model when offline)
- Add a recency filter using Week 1 metadata
- Score the 20 judged questions in data/golden

Measure and report

- nDCG@10 at four stages: dense only, hybrid, hybrid plus rerank, plus filter
- Which of last week's failed queries now work
- Added latency at each stage
- One query that got worse, and your theory why

Lab debrief

1



The four numbers

Two pairs share their nDCG at each stage.

Where was the biggest jump? It is usually reranking, but not always.

2



The rescued queries

Which Week 1 failures now work?

Read two aloud. This is the visible payoff of the week.

3



The one that got worse

Collect these.

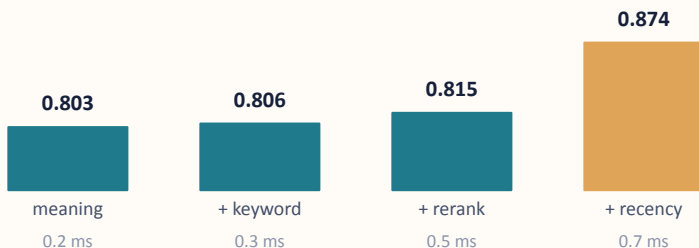
Usually a filter that was too aggressive, or a reranker mismatched to jargon.

Keep your judged query set. It becomes the evaluation set for grounding tomorrow and for the capstone in Week 6.

What each stage bought, and what it cost

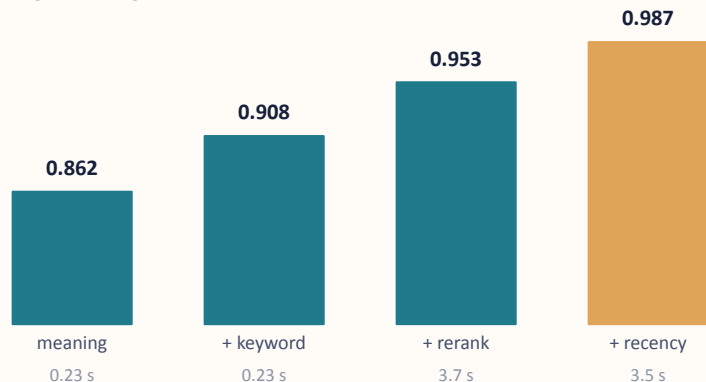
Offline track

mean nDCG@10 over 20 questions, time per question; bars start at 0.75



Real models on Databricks

gte-large embeddings, Llama 3.3 70B reranker; bars start at 0.75



Offline, reranking made R02, R04 and R05 worse. With real models nothing got worse, and R17 went from 0.00 to 1.00 once the scanned PDF was read.

ASK THE ROOM Which stage would you ship for Priya on a live chat, knowing reranking adds about 3.5 seconds?

Same stack, two failures, two different fixes

	MEANING ONLY	+ KEYWORD	+ RERANK	+ RECENTY
R01 “how many refund claims can a customer make in thirty days” A data problem. The 2022 policy says two, the 2024 policy three; the words are almost identical.	0.579 answer 2nd, withdrawn policy 1st	0.579 keyword search makes the same mistake	0.579 the reranker reads the same words	0.917 withdrawn chunks removed: answer 1st
R02 “who pays when food is spilled in transit” A vocabulary problem. The answer says “packaging inadequate”, not “food spilled”.	0.311 answer 6th	0.311 keyword finds it 5th; merged, still 6th	0.262 fewer shared words, so the answer drops to 9th	0.333 answer still 6th: still failing

R01 is fixed by better data, R02 only by better models: 0.88 on meaning search alone with real models.

ASK THE ROOM Pick one question still failing in your run. Is it R01's kind of failure, or R02's?

Three things to take from today



Two methods, not one

Dense and keyword fail in opposite directions.
Run both, fuse by rank.



Wide then narrow

Retrieve for recall, rerank for precision. Stage 1 sets the ceiling stage 2 cannot exceed.



Filters are correctness

Recency and entitlement are enforced at retrieval. A chunk in the context window is disclosed.

Tomorrow: your system retrieves well but still cannot be trusted, because it will answer confidently even when the retrieved chunks do not contain the answer. We fix that with grounding and citations.

AI ENGINEER · DATA · WEEK 2, DAY 4

Grounding & Citations

and knowing when to say nothing at all

4 hours · instructor-led · build a RAG API

Yesterday in one minute

- Dense and keyword retrieval fail in opposite directions, so run both
- Retrieve wide, rerank narrow. Stage 1 sets the ceiling
- Recency and entitlement filters are correctness controls
- nDCG, not recall, is what moves when you rerank

Today's uncomfortable fact

Your retrieval is now good. Your system will still answer confidently when the right chunk was not found.

Better retrieval reduces how often that happens. It does nothing about what happens when it does.

Retrieval quality and answer trustworthiness are two different problems. Today is the second one.

Where we are going today

1

Grounding

Answer only from the chunks you retrieved, and say so when the answer is not there.

2

Citations

Point every claim at its source, and check that pointer in code before anyone sees it.

3

Measuring trust

Faithfulness, refusal, and why one accuracy number hides the failure.

4

Lab 4

An answer service that proves where every answer came from.

Two 10-minute breaks. The lab starts after section 3, with 80 minutes and a protected debrief.

BY THE END OF TODAY Priya gets an answer with a source she can paste to the customer, or an honest "I could not find this".



1

Grounding

Answer only from what you retrieved, and say so when it is not there



An analogy

The witness who never says "I don't know"

Imagine a witness with perfect recall of everything they have ever read, an eagerness to be useful, and one flaw: they are constitutionally incapable of saying "I was not there".

Ask what happened on Tuesday and they will tell you, fluently, in the right register, with plausible detail. If they were there, it is accurate. If they were not, it sounds exactly the same.

You cannot fix this by asking them to be more careful. You fix it by only asking about things you have handed them evidence for, and by making them point at the evidence for every claim.

That is grounding, and that is why citations are a control rather than a courtesy.

A chatbot invented a refund rule. Air Canada paid.

November 2022

Jake Moffatt's grandmother dies. He asks the chatbot on Air Canada's website about bereavement fares.

The answer

Book now at the full fare, and claim the bereavement discount within 90 days. He does.

The real policy

Bereavement fares cannot be claimed after travel. Air Canada refuses the refund.

February 2024

A British Columbia tribunal rules for him. Air Canada is responsible for everything on its website, chatbot included. It pays about CAD 812.

Air Canada argued the chatbot was a separate legal entity, responsible for its own actions. The tribunal rejected that.

WHAT WOULD HAVE STOPPED IT Answer only from the retrieved policy page. Cite it, so the answer can be checked. If the policy does not say it, refuse. That is today, in three sentences.

ASK THE ROOM FreshCart's assistant quotes a claim limit that is in no policy. Who pays the customer?

Grounding, in one sentence

Making the model answer only from the chunks you retrieved, and making it show which chunk each claim came from.

- Two halves: constrain what it may use, and make the link between claim and source visible
- Without the second half you cannot tell a grounded answer from an invented one, both read the same
- Citations are how a support agent, an auditor, or a regulator checks the system without trusting it

A RAG answer without a checkable citation is a rumour with good grammar.

Why models hallucinate



Nothing relevant retrieved

The answer was not in the context. The model fills the gap from training memory.

Fix: retrieval, plus a refusal path when nothing scores well.



Conflicting chunks

Two retrieved documents disagree, often an old and a new policy.

Fix: recency filters and metadata, plus telling the model how to handle conflict.



Instructed to be helpful

The prompt rewards a complete answer, so it produces one.

Fix: explicitly permit and reward "I could not find this".

Only the first is a retrieval problem. The other two are design problems, and no amount of embedding tuning touches them.

Hallucination: the model states something the sources do not support, as confidently as something they do. It can invent a fact, or invent a source.

The grounding prompt from Lab 4, line by line

GROUNDING_PROMPT

- 1 Answer using only the provided context.
- 2 If the context does not contain the answer, say you could not find it.
- 3 Cite the source id in square brackets for every factual claim.
- 4 If sources disagree, say so and prefer the most recent.
- 5 Do not infer beyond what the context states.

WHY EACH LINE IS THERE

- Removes the model's memory as a source
- Makes "not found" a success, not a failure
- Creates a pointer your code can check
- Stops a silent pick between the 2022 and 2024 policies
- Cuts plausible extrapolation

WHAT CAME BACK "A customer may make up to three refund claims in any rolling thirty day period ... [37801390e45c]". One fact, one citation, from the 2024 policy.

ASK THE ROOM Delete line 2. What changes in your refusal rate, and would anyone notice?



2

Citations

A pointer to the source, checked in code before anyone sees it



What a citation has to be



Specific

Points to a chunk or a passage, not a 400-page document.

"See the HR policy" is not a citation.



Resolvable

The user can open the source and see the sentence.

Requires the provenance metadata you attached in Week 1.



Verified

Your code checks the cited chunk was actually retrieved and actually contains the claim.

Models do cite chunks they did not use.

An unverified citation is worse than none. It creates the appearance of auditability while providing none of it.

A made-up source looks exactly like a real one

What Priya sees, pasted from the answer service into her reply to the customer:

ANSWER

A customer may make up to three refund claims in any rolling thirty day period [37801390e45c]. A fourth claim in that period is declined automatically [9f3a0c2b71de].

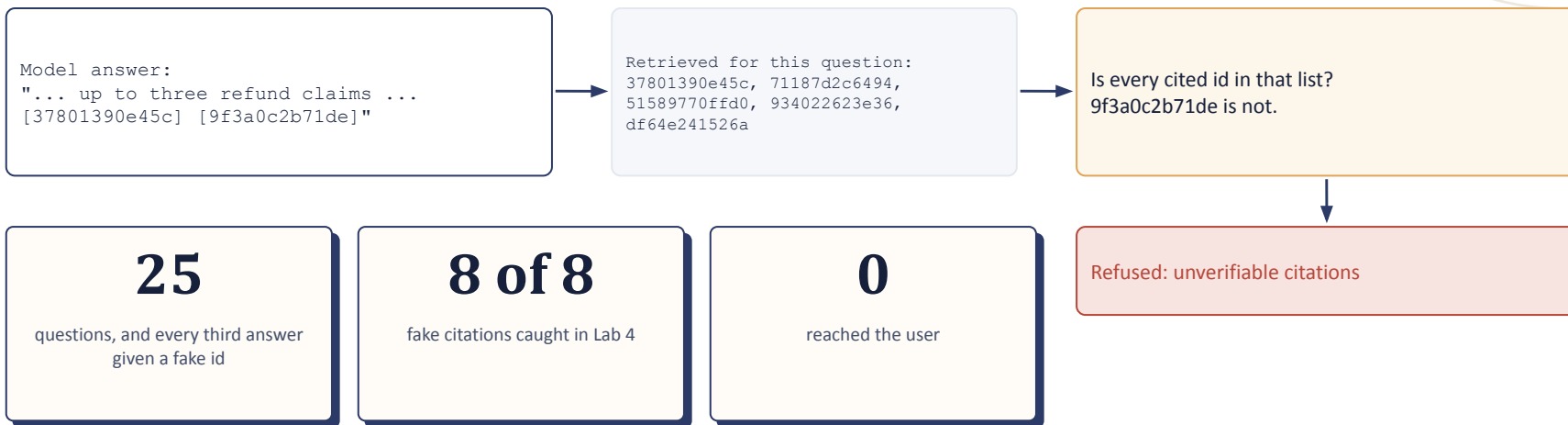
Citation 1: [37801390e45c] Same format, same place, same confidence.

Citation 2: [9f3a0c2b71de] Same format, same place, same confidence.

One of these two sources was never retrieved for this question. Nothing on the page tells you which.

ASK THE ROOM Which citation would you trust? How would you find out before the customer does?

The check that catches a citation to nothing



WHAT THE CHECK DOES NOT PROVE A real id can still point at a chunk that does not support the sentence. The check catches invented sources; supported sentences (faithfulness), after the break, catch unsupported claims.

ASK THE ROOM Why not just ask the model to double-check its own citations?

Before the break, three questions

1

A user asks a question whose answer genuinely is not in your corpus. Describe what a well-designed system does, step by step.

2

Your answers all carry citations and users trust the system. An auditor picks one answer at random. What would you need to have built for it to survive that check?

3

Your system suddenly refuses to answer 40 percent of questions after a change. Give two plausible causes.

Take answers in chat. Do not move on until at least two people have committed to an answer.

Before the break, three questions

1

The answer is not there

Retrieval returns candidates; every one scores below the match-score threshold; the system declines to answer, says what it searched, and offers a next step such as escalation. It does not produce a fluent guess. Refusal is a success state.

2

Surviving an audit

Provenance metadata from ingestion, chunk-level citations, a stored record of which chunks were retrieved and sent, verification that cited chunks were actually in that set, and the document version that was current at answer time.

3

Sudden mass refusal

Any two of: a match-score threshold set too high; a filter that is too aggressive, often an entitlement or recency filter excluding almost everything; a retrieval regression such as a broken index or changed embedding model; or a prompt change that over-weighted caution.

Break

10 minutes. After the break: how to prove your answers are actually grounded.



3

Measuring trust

Faithfulness, refusal, and the limits of automated judging



Three different questions about one answer



Supported sentences

Faithfulness: is every claim supported by the retrieved chunks?

Catches invention. This is the grounding metric.



Answers the question

Answer relevance: does the answer address what was actually asked?

A faithful answer to the wrong question is still useless.



Right pieces

Context relevance: were the retrieved chunks the right ones?

This is yesterday's work, measured from the answer end.

A system can score well on one and badly on another. Reporting a single "accuracy" number hides which part is broken.

Fully grounded, correctly cited, and no use to Priya

A14 "What is the first response TAT for a P2 case?"

THE ANSWER

FreshCart Support SLA Matrix [934022623e36]

WHAT IT SHOULD SAY

30 minutes

Supported sentences

Faithfulness: is every claim supported by the retrieved text?

yes, 1.000

Real citations

Citation check: was the cited chunk actually retrieved?

yes

Answers the question

Answer relevance: does it answer the question asked?

no

WITH THE FREE ANSWER WRITER Supported sentences (faithfulness) are 1.000 across the board, and 3 of 20 answers still miss the fact asked for. The real model in Lab 4 states all 3; the stored facts are what prove it.

ASK THE ROOM A14 failed. Was it retrieval, chunking or the answer writer? Defend one.

Supported sentences: did each one come from the retrieved text?

1 Split the answer into sentences

2 In each sentence, take the words of 4 or more letters

3 A sentence is supported if at least 60% of those words appear in the 5 retrieved pieces

4 Score = supported sentences ÷ all sentences, from 0 to 1

LAB 4, REFUND QUESTION one sentence

"A customer may make up to three refund claims in any rolling thirty day period [37801390e45c]."

customer

make

three

refund

claims

rolling

thirty

period

8 of 8 words found = 100% **supported**

1 of 1 sentence supported **supported sentences 1.000**

LAB 4, U01: GROCERY REFUNDS IN SINGAPORE two sentences

"I could not find the answer."

could

find

answer

0 of 3 = 0% **not supported**

"The provided context only discusses the refund policy ..."

7 of 15 words found = 47%, under 60% **not supported**

Found: refund, policy, food, delivery, orders (twice), india. Missing: provided, context, only, discusses, does, mention, grocery, singapore.

0 of 2 sentences supported **supported sentences 0.000**

WHY Supported sentences (faithfulness) check where the words came from, not whether the answer is right or useful. A14 scores 1.000 and never states the 30 minutes; U01's honest "I could not find it" scores 0.000, because those words are not in any document.

ASK THE ROOM Is a low supported-sentences score always bad news? Use U01 to argue it.

The match score: how well the best piece's words match

What it is The reranker's score for each of the 5 retrieved pieces, about 0 to 1. The service returns the best one as the confidence field, but it is not how sure the model is. Code works it out; no answer key.

LAB 4, REFUND QUESTION best piece: the claim limits section of the refund policy

Question words found in the piece	rare words count more	0.656×0.8	0.525
How close together those words sit	1 if they are side by side	0.584×0.2	0.117
Word match		0.642×0.7	0.449
Place in the Lab 3 search list	1st place = 1, 2nd = 0.5, 3rd = 0.33	1.000×0.3	0.300
$0.449 + 0.300 =$ match score 0.749			

U01: NO ANSWER IN THE DOCUMENTS

"What is FreshCart's refund policy for grocery orders in Singapore?"

found in the best piece

freshcart refund policy order

missing

grocery singapore

The two missing words are the two that decide the answer.

match score 0.740 almost the same

WHY The score counts matching words and never reads for meaning, so a question the documents cannot answer can score as high as one they can. Use it to refuse clearly hopeless questions before paying for a model call, not to decide whether an answer exists.

ASK THE ROOM The service calls this number "confidence". Confidence in what, exactly?

What each score checks, and what it can miss

	Ranking quality nDCG@10, Lab 3	Match score retrieval score, confidence, Lab 4	Supported sentences faithfulness, Lab 4	Right fact correct facts, Lab 4
Asks	Are the useful pieces near the top?	Do the best piece's words match the question?	Did each sentence come from the retrieved text?	Does the answer state the fact asked for?
Checked against	0 to 3 grades a person wrote	Nothing: code works it out	The 5 retrieved pieces	The expected phrase stored per question
Answer key needed	Yes	No	No	Yes
Runs	Testing only	On every live question	On every answer	Testing only
Fooled by	Only covers questions someone graded	U01 scores 0.740 with no answer	A14 scores 1.000 and misses the fact	A right answer in other words

SO The two that need an answer key are the honest ones, and they only run in testing. The two that run live on every question need no key and are easier to fool. A service needs both kinds, reported side by side.

ASK THE ROOM Priya's team can afford one score on every live answer. Which one, and what do they still test before release?

Three checks on citations, and what Lab 4 found

Real citations

citation check

Was every cited piece actually retrieved?

8 of 8 fake citations caught

Low means: Invented sources

Live: refuses the answer

Cites only what it used

citation precision

Of the pieces cited, how many did the answer use?

20 of 22 cited pieces used, 0.91

Low means: Extra citations added to look safe

Step 6: reported only

Every claim cited

citation recall

Of the sentences that state something, how many carry a citation that supports them?

19 of 19 claims cited and supported, 1.00

Low means: Facts with no proof of where they came from

Step 6: reported only

A16, THE ONE PRECISION MISS "According to [b0563f97ea12] and [52f5f2a59ffe], FreshCart bears the full cost of the refund when the failure arises from delivery partner handling."

The second piece holds the end of the same contract clause, which chunking cut in two. Only 54% of the sentence's words are in it, under the 60% bar, so the rule counts it as unused. Arguably a fair citation: the rule is a word check, not a judge.

SO Only the first check refuses answers. The other two are cheap word checks that raise false alarms like A16, so report them and read the cases. Recall is perfect here because answers are one sentence long.

ASK THE ROOM Which of the three would you let refuse an answer in production, and why only that one?

Building the evaluation set

1



Start from real questions

Support tickets, search logs, the failed queries from Week 1. Not questions you invented.

2



Write the expected answer

And the chunk that should support it. This is the slow part and the valuable part.

3



Include unanswerables

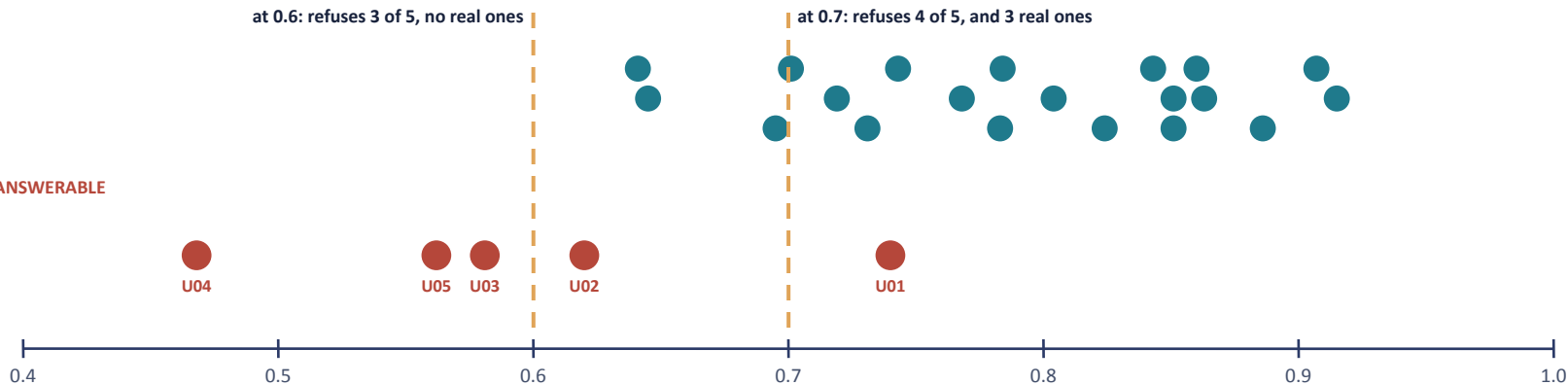
Questions your corpus genuinely cannot answer. The correct output is a refusal, and you must test for it.

Fifty questions, judged carefully, with ten of them unanswerable, will tell you more than five hundred generated by a model from your own documents.

No single threshold separates the two groups

20 ANSWERABLE

5 UNANSWERABLE



WHY A match score (retrieval score) says how similar the text is, not whether it answers the question. U01, "refund policy for grocery orders in Singapore", scores 0.740: very similar to the refund policy, and still unanswerable. The fix is a rule the answer writer must follow, which Lab 8 puts into code.

ASK THE ROOM Where would you set the threshold for Priya, and who gets hurt by your choice?

Using a model to judge



Where it works

Checking whether a claim is supported by a given chunk. A narrow, verifiable comparison.

It scales to thousands of judgements cheaply.



Where it drifts

Open-ended quality judgements. It favours fluent, longer answers and shares blind spots with your answering model.

It cannot judge domain correctness it never learned.



How to keep it honest

Human-judge a sample regularly and compare. If they diverge, trust the human and recalibrate.

Automated judging tracks change over time well. It is a poor substitute for having decided what good looks like.

Evaluation tools: what each one measures

TOOL	WHAT IT SCORES	WHERE IT RUNS
RAGAS	Faithfulness, answer relevancy, context precision and recall	Open source, Python
TruLens	The RAG triad: context relevance, groundedness, answer relevance	Open source, dashboard
DeepEval	The same checks, written as unit tests that pass or fail	Open source, pytest style
MLflow	Model judges logged with every run, so scores compare across versions	Built into Databricks; Lab 10
Azure AI Foundry	Groundedness and relevance evaluators over your test set	Managed, Azure

BEFORE YOU TRUST ANY OF THEM All of them use a model as the judge. Grade 20 answers by hand, run the tool on the same 20, and compare. Lab 4 uses the simplest possible judge, word overlap in `src/rag.py`, so you can see exactly what it checks.

ASK THE ROOM Which of the three questions (faithful, relevant, right context) would your stakeholders notice first?

One answer service, many callers, the same proof every time

WHO CALLS IT

Priya's support screen

shows the answer and its source

A chat window

for customers or new agents

An agent

uses it as one of its tools

A nightly test (Lab 10)

runs the 25 questions, counts

THE ANSWER SERVICE

- 1 Find the 5 best pieces (Lab 3)
- 2 Match score too low? Refuse, no model call
- 3 The model writes the answer with citations
- 4 Every citation retrieved? If not, refuse
- 5 Return six fields

WHAT EVERY CALLER GETS BACK

answer	the text, with [ids]
citations	ids, checked in code
confidence	match score of the best piece
retrieved	the 5 pieces sent to the model
refused	true or false
refusal_reason	why, in words

Proof with every answer

The ids let anyone check the source. A plain chatbot gives a sentence and nothing else.

Safe failure, with a reason

A refusal tells the caller why, so it can hand over to a person instead of guessing.

One place for the rules

Threshold, prompt and citation check live once. Every caller gets the same protection.

Testable by a script

Structured fields mean a script can score all 25 questions every night.

ASK THE ROOM Three teams each call the model directly from their own app. What goes wrong first?

What the answer service returns, field by field

```
{  
  
  "answer": "... up to three refund claims  
  
    in any rolling thirty day period ... [37801390e45c]",  
  
  "citations": ["37801390e45c"],  
  
  "confidence": 0.749,  
  
  "retrieved": ["37801390e45c", "71187d2c6494",  
  
    "51589770ffd0", "934022623e36", "df64e241526a"],  
  
  "refused": false,  
  
  "refusal_reason": null  
}
```

answer

The text, or null when refusing. Never a made-up fallback.

citations

Ids that the code has checked were retrieved.

confidence

The match score of the best piece: lets the caller decide when to escalate.

retrieved

Everything that was sent to the model. Makes any answer reproducible.

refused, refusal_reason

"Not found" and "not permitted" look the same unless you say which.

ASK THE ROOM An auditor picks this answer six months from now. Which fields let you prove what happened?

Three questions before the lab

1

Your model returns an answer citing chunk 12, but chunk 12 was never retrieved for that query. How would your code catch this, and what should it do?

2

Faithfulness scores 0.95 and users still complain. What is the most likely explanation?

3

You add "if you are unsure, say you do not know" to the prompt and refusals jump to 60 percent. What went wrong, and how would you diagnose it?

Take answers in chat. Do not move on until at least two people have committed to an answer.

Three questions before the lab

1

The phantom citation

Keep the set of chunk ids you actually sent, and assert every cited id is in that set. If not, treat the answer as ungrounded: drop the citation, and either regenerate or refuse. Never surface a citation you could not verify.

2

High faithfulness, unhappy users

Almost certainly answer relevance. The system is faithfully summarising chunks that do not address the question, usually a retrieval or reranking problem, not a generation one. Faithfulness alone cannot detect this.

3

The refusal spike

The prompt made caution cheap and being wrong expensive, so the model refuses when it should answer. Diagnose by running the unanswerable subset of your eval set separately from the answerable one: if refusals rose on answerable questions, the prompt is miscalibrated, not the retrieval.

Day 4: before you build the service

Does grounding stop hallucination?

It stops invention from memory. It does not stop answering when the documents are silent: with a free writer the service answers all 5 unanswerable questions, and with a real model it still lets 1 through.

Can we set a match-score threshold and be done?

No single value works. Answerable questions score 0.641 to 0.915, and unanswerable ones 0.468 to 0.740. The two ranges overlap.

Why check citations in code? The model wrote them.

Models cite chunks they were never given. The check is one line: every cited id must be in the list you sent. Lab 4 catches all 8 fake ones.

Faithfulness is 1.000. Are we done?

No. With a free writer, 3 of 20 fully grounded answers still miss the fact that was asked for. Store the fact each answer must contain, and check for it.

Is a model judging a model circular?

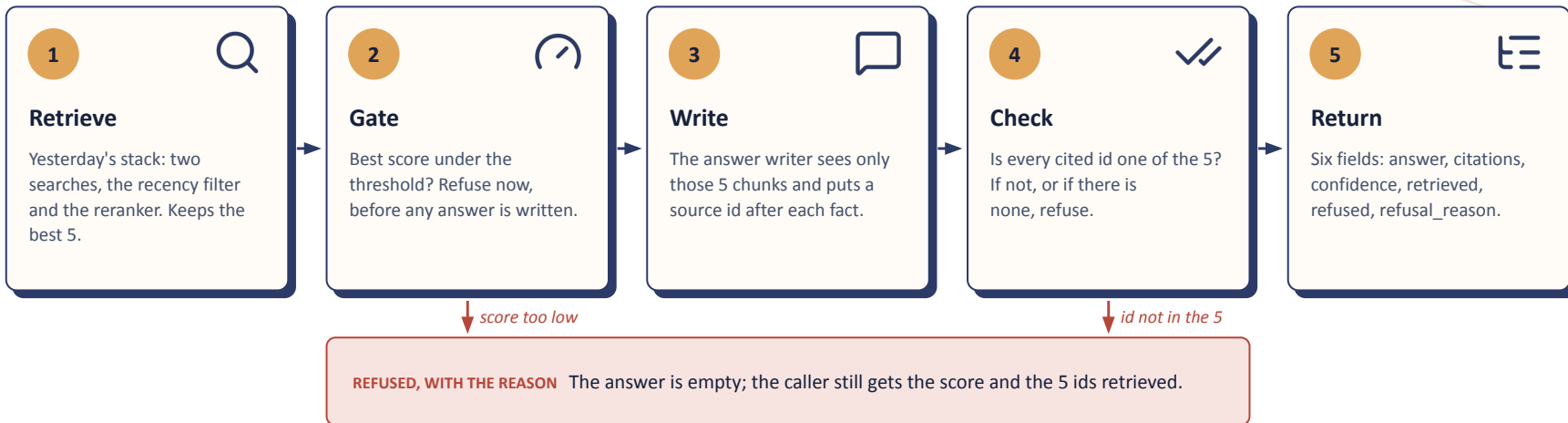
Partly. It is reliable for narrow checks, such as whether a claim is in a chunk, and for tracking change over time. Grade a sample by hand and compare.

What do we keep for the auditor?

The question, the ids retrieved and cited, the answer, the refusal reason and the document version. The six-field response is most of that record.

One more, and it decides who trusts the system: what does it do when the answer is not there? Decide that before you tune anything else.

One question through the answer service



YOU WILL BREAK IT ON PURPOSE Every third answer gets a fake source id added. The text still looks normal, so only step 4 can catch it.

ASK THE ROOM Step 2 or step 4: which refusal would you rather see more often, and why?

One /ask call, step by step, and the code that runs each step

THE CALLER SENDS

POST /ask

```
{
  "question":
    "How many refund
     claims ... thirty
     days?",
  "threshold": 0.15}
```

ALSO

```
GET /health
answers {"status": "ok",
"chunks": 30} when the
service is up
```

1	Read the request FastAPI checks the JSON; threshold defaults to 0.15	labs/lab04_rag_api.py
2	Find candidates meaning + keyword search, merged by rank; recency filter drops old policies	src/retrieval.py
3	Rerank, keep 5 a match score per piece: 0.749, 0.379 ... 0.266	src/retrieval.py
4	Gate best match score under the threshold? stop, no model call	src/rag.py
5	Write grounding prompt + 5 pieces tagged with ids + question	answer writer
6	Check citations every [id] must be one of the 5 retrieved	src/rag.py

Step 6 also refuses when there is no citation at all. A refusal still returns all six fields.

THE CALLER GETS BACK

```
{
  "answer": "... three
  refund claims ...
  [37801390e45c]",
  "citations":
    ["37801390e45c"],
  "confidence": 0.749,
  "retrieved": [5 ids],
  "refused": false,
  "refusal_reason": null}
```

RUN IT python labs\lab04_rag_api.py serve starts it on the Windows VM. Open <http://127.0.0.1:8000/docs>, choose POST /ask, Try it out, Execute. A Databricks notebook cannot open a port, so the class notebook stops before this.

ASK THE ROOM Send the Singapore grocery question with threshold 0.75. Which step stops it, and what comes back?

Lab 4

Ship a RAG API with verifiable citations

80 minutes · same pairs · lab04_rag_api notebook

Build

- An endpoint over yesterday's retrieval stack
- Grounded prompt with explicit refusal permitted
- Chunk-level citations in the response
- Code that verifies every cited id was retrieved
- A match-score threshold below which you refuse
- The full response contract, all six fields

Prove it works

- 20 answerable and 5 unanswerable questions
- Supported sentences (faithfulness) on the answerable set
- Refusal rate on each set, reported separately
- At least one caught phantom citation
- One answer the code read wrongly

Same service, two answer writers

	FREE WRITER	REAL MODEL
Refused, answerable (want low)	0 of 20	1 of 20
Refused, unanswerable (want high)	0 of 5	4 of 5
Supported sentences (faithfulness)	1.000	1.000
Right fact (correct facts)	17 of 20	19 of 20
Fake citations caught	8 of 8	8 of 8

Free writer: quotes the top chunk, offline track. Real model: Llama 3.3 70B on Databricks, about 25 calls, a few cents a run.

BACK TO PRIYA The policy half of her question now comes with a source. "How many claims has this customer made this month?" is a number, and documents cannot give it. Week 3 goes to the warehouse.

ASK THE ROOM Refusals on answerable questions jump from 0 to 6 after a prompt change. What do you check first?

Two right answers, misread by the code around the model

A15 First response TAT for a food safety complaint?

THE MODEL WROTE

```
The first response TAT for a food safety complaint is 15 minutes  
[88ddf531a0f2, 934022623e36].
```

WHAT THE SERVICE'S CODE READ

The reader looks for one id alone in a bracket, so two ids in one bracket count as none: [].

today: **REFUSED**, no citation produced

Fix: read every id inside any bracket. Both were retrieved.

with the fix: **ACCEPTED**, the right answer

U01 Refund policy for grocery orders in Singapore?

THE MODEL WROTE

```
I could not find the answer. [60245dc902e1] states that the refund  
policy applies to food delivery orders in India ...
```

WHAT THE SERVICE'S CODE READ

The reader finds a valid id, and nothing checks the words. The answer says no; the fields say yes.

today: **ACCEPTED** as an answer

Fix: treat "could not find" as a refusal, whatever else the answer contains.

with the fix: **REFUSED**, as it should be

SO The model did the right thing both times. With a good model, the bugs move into the code around it. Print what the model wrote next to what the service decided (Step 5), then run the checking code on that text (Step 5b).

ASK THE ROOM Which fix would you ship first, and what test stops the bug coming back?

Week 2 in three sentences



You found the right chunk

Two searches merged by rank, a reranker and a recency filter took nDCG@10 from 0.803 to 0.874 offline, and from 0.862 to 0.987 with real models on Databricks.



You can show where it came from

Every answer carries source ids that code has checked. 8 fake citations went in, and none reached the user.



You know what is still unsafe

With a free answer writer the service answered all 5 questions the documents cannot answer, and with a real model it still let 1 through. No threshold separates them cleanly.

Week 3: the questions documents cannot answer. “How many claims has this customer made this month?” is a number, so we go to the warehouse: questions turned into SQL, and one agreed definition for every metric.